
COMPLEX ENGINEERING PROBLEM

TITLE

Open-Source Repository Documentation Improvement

Jainish. Kakkad [92400103550]

Jay. Visana [92400103529]

Parth. Hirani [92400103541]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026-2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS

EC 6

BATCH

B

COMPLEX ENGINEERING PROBLEM - REPORT

RUBRICS FOR COMPLEX ENGINEERING PROBLEM

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis & Open Source Technology Selection	3 Marks: Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate Open Source technologies with proper justification.	2 Marks: Good problem analysis with suitable technology selection and minor gaps.	1 Mark: Basic analysis with limited justification of technology selection.	0 Marks: Inadequate problem analysis and inappropriate technology selection.	3
2	System Design, Implementation Strategy & Technical Analysis	4 Marks: Well-defined system architecture, appropriate technology stack, database, deployment strategy, version control, and comprehensive technical analysis including scalability, reliability, and security.	3 Marks: Good system design and technical discussion with minor omissions.	1-2 Marks: Basic architecture and limited technical analysis.	0 Marks: Poor or missing system design and technical analysis.	4
3	Testing, Documentation & References	3 Marks: Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.	2 Marks: Good documentation with minor formatting or testing gaps.	1 Mark: Basic report with limited testing or references.	0 Marks: Incomplete report with poor documentation and missing references.	3
	Total					10

Sr. No.	Complex Engineering Problem Activity / Problem Title	COs	Date	R1(4)	R2(3)	R3(4)	Marks (10)	Signature
1	Open-Source Repository Documentation Improvement							

Ms. Priyanka Mangi
Subject Faculty

COMPLEX ENGINEERING PROBLEM - REPORT

TITLE: OPEN-SOURCE REPOSITORY DOCUMENTATION IMPROVEMENT

Course Name: OPEN-SOURCE TECHNOLOGIES (01CE0526)

COs Mapped: CO1

SDGs Mapped: SDG 4, SDG 9, SDG 17

1. Aim

To improve the documentation and contribution structure of the OpenShelf open-source repository by connecting a functioning browser-based library application with clear documentation, contributor guidance, issue reporting, version-control conventions, automated validation, and a reviewable GitHub workflow.

2. Problem Statement

An open-source library repository needs accurate documentation and a repeatable contribution process so that users can understand the application, contributors can reproduce the setup, changes can be reviewed, and repository quality can be checked consistently. OpenShelf therefore required a structured documentation and contribution workflow around a working library application.

3. Problem Analysis

a) What is the engineering problem?

The engineering problem is to organize a functioning library application repository so that application source code, documentation, contribution rules, issue reporting, version control, testing, and continuous integration remain consistent, traceable, and maintainable.

b) Why is this problem complex?

- Requires multiple engineering concepts.
- Requires analysis and decision making.
- Involves design constraints and repository organization.
- Requires use of engineering tools.
- Requires performance and security consideration.

4. Knowledge and Tools Required

Engineering concepts: open-source development, Git version control, repository organization, documentation engineering, client-side web development, browser storage, automated testing, continuous integration, and collaborative change review.

Tools / Software / Hardware: Git, GitHub, GitHub Issues, GitHub Pull Requests, GitHub Actions, HTML, CSS, JavaScript, Node.js, npm, Markdown, browser, code editor, and computer/laptop.

Open-Source Technology Selection

Technology	Role	Reason for selection
HTML5 + CSS3	Interface	Portable structure and separated presentation.
JavaScript	Application logic	Implements catalogue operations and UI behaviour.
Git + GitHub	Version control	Provides traceable history, branches, issues, and pull requests.
Node.js + npm	Validation	Runs repeatable local checks.
GitHub Actions	CI	Repeats validation automatically on repository events.

COMPLEX ENGINEERING PROBLEM - REPORT

5. Proposed Solution Design

The proposed repository combines a small browser-based library application with structured open-source collaboration practices. index.html provides the interface, js/app.js controls catalogue state and events, css/styles.css provides responsive presentation, and browser localStorage persists catalogue and activity state. GitHub hosts the source and documentation, while automated checks validate the repository.

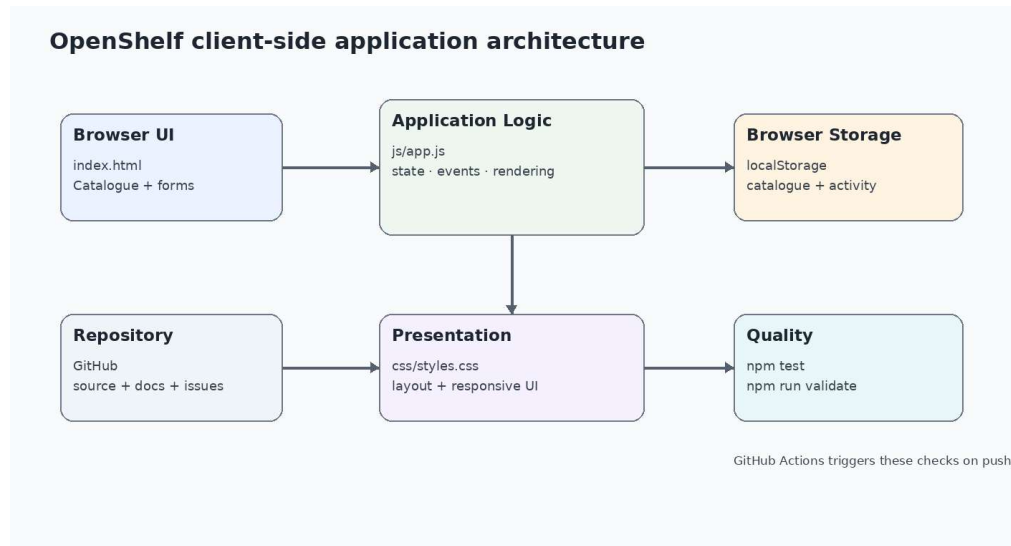


Figure 1 – OpenShelf client-side application architecture



Figure 2 – Repository documentation improvement workflow

Brief Working

The workflow begins with a documented GitHub issue. The documentation change is isolated on docs/library-workflow, applied to README.md, committed with a focused message, and proposed through Pull Request #2. The pull request exposes the exact diff, triggers automated validation, and is merged after the checks pass and no conflicts remain.

COMPLEX ENGINEERING PROBLEM - REPORT

6. Implementation / Procedure

- Step 1:** Scaffolded the OpenShelf library application and committed the baseline.
- Step 2:** Added searchable catalogue behaviour and category filtering.
- Step 3:** Added issue/return tracking, catalogue maintenance, and browser persistence.
- Step 4:** Added README, contribution guidance, issue templates, tests, and repository validation.
- Step 5:** Created GitHub Issue #1 describing the need for clearer library workflow documentation.
- Step 6:** Created docs/library-workflow and updated README.md.
- Step 7:** Committed the documentation change and opened Pull Request #2 linked to Issue #1.
- Step 8:** Passed automated checks and merged the reviewed change into main.

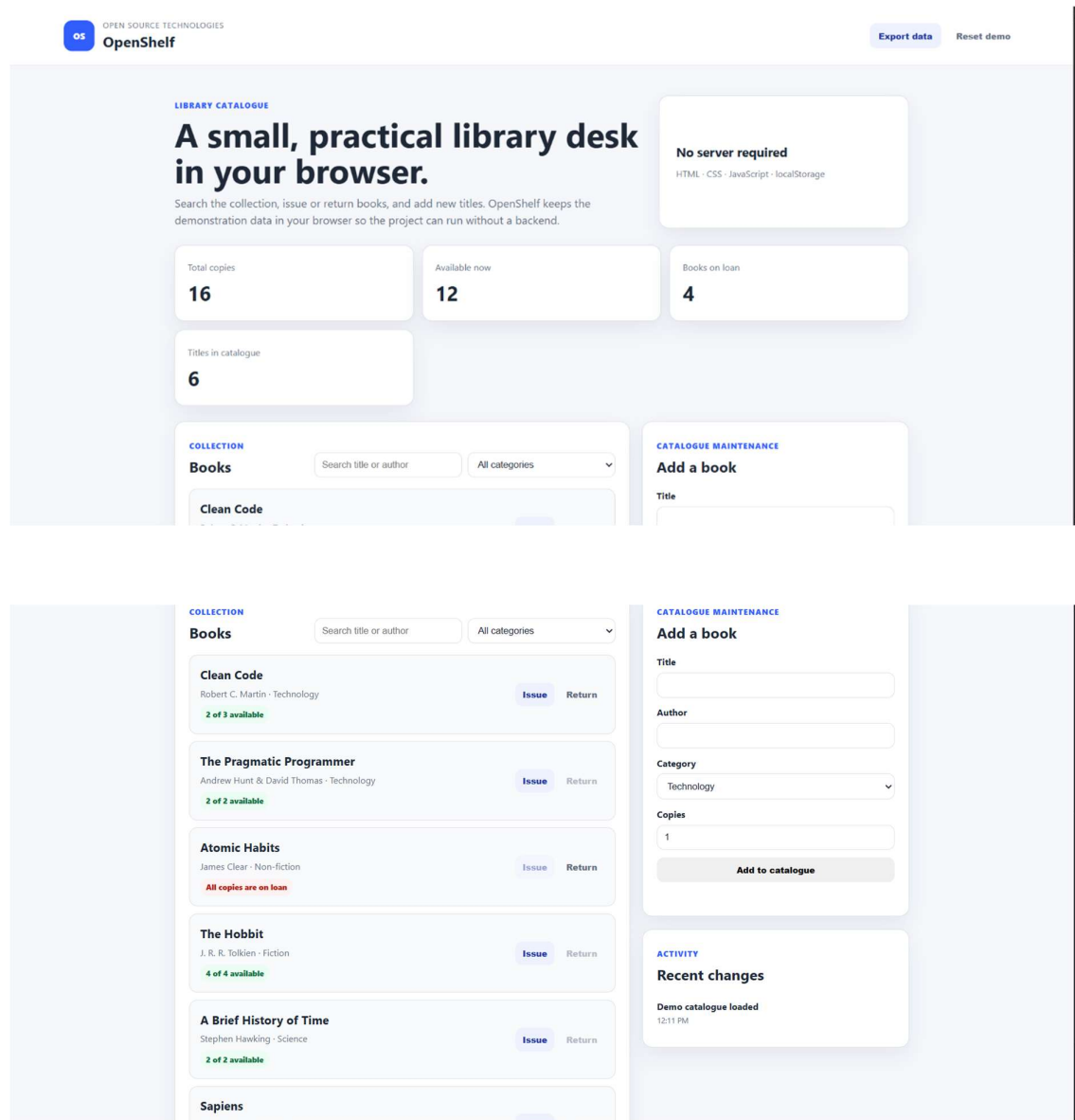


Figure 3 – OpenShelf library catalogue and maintenance interface

COMPLEX ENGINEERING PROBLEM - REPORT

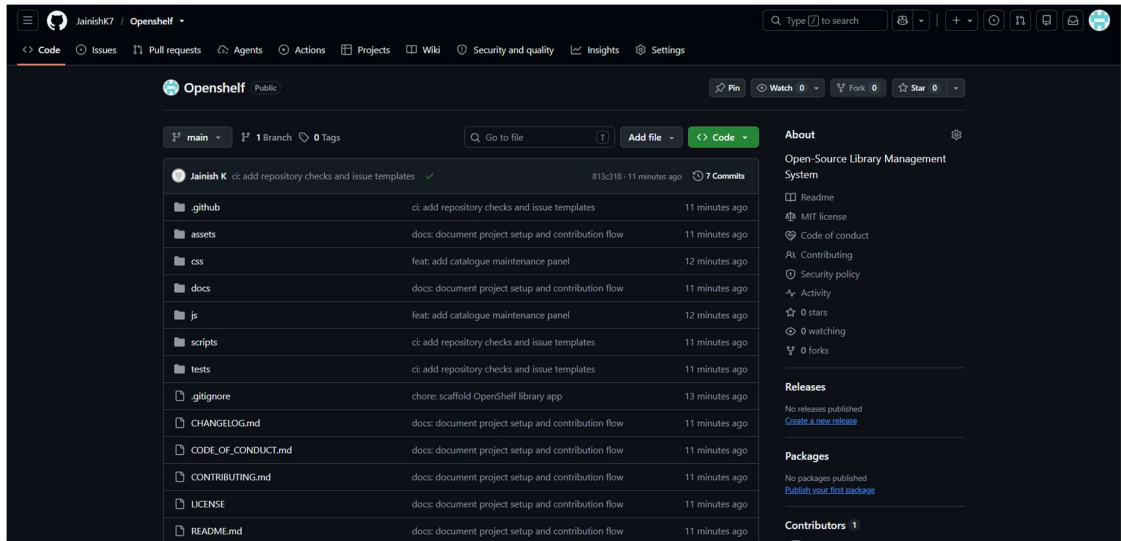


Figure 4 – OpenShelf repository baseline and project structure

COMPLEX ENGINEERING PROBLEM - REPORT

6.1 Issue and Branch Evidence

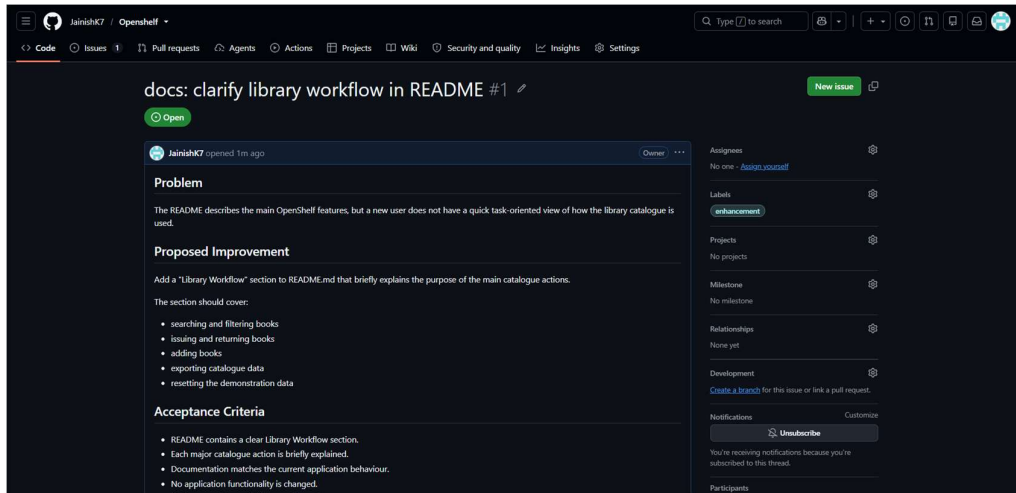


Figure 5 – GitHub Issue #1 documenting the README improvement

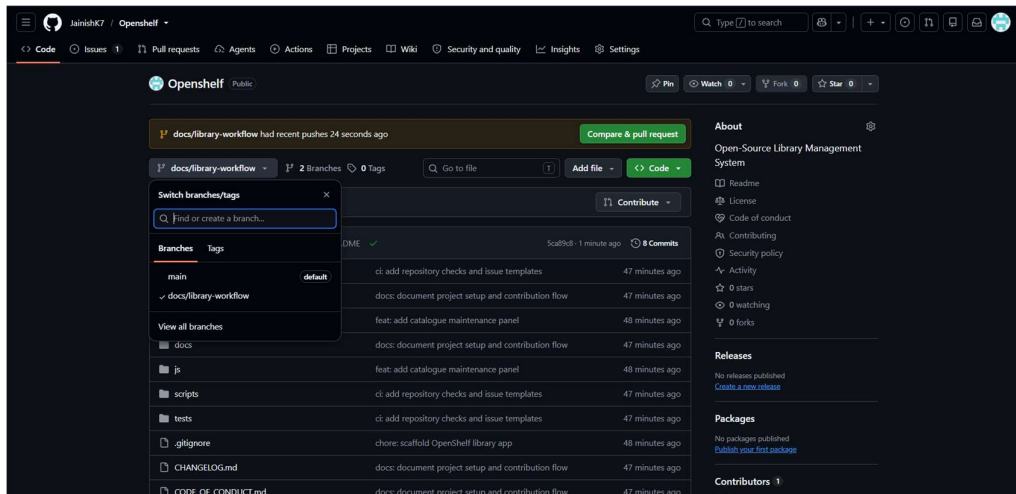


Figure 6 – docs/library-workflow branch

Issue #1 records the documentation gap and its acceptance criteria before implementation. The dedicated documentation branch isolates the work from main and provides a traceable location for the README update.

COMPLEX ENGINEERING PROBLEM - REPORT

6.2 Pull Request and Review Evidence

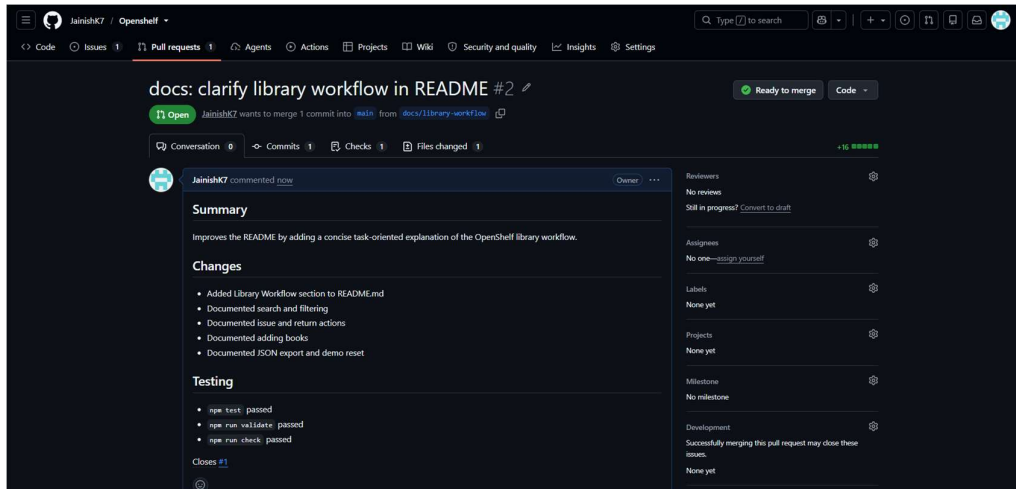


Figure 7 – Pull Request #2 for the README improvement

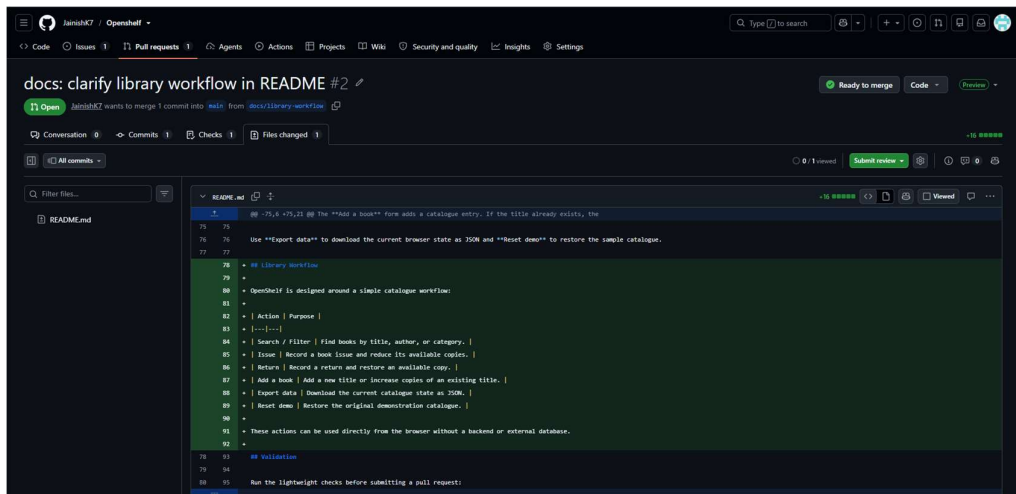


Figure 8 – README diff reviewed through Pull Request #2

The pull request links the issue to the implementation and presents the exact documentation diff for review. The change is intentionally focused on README.md and does not alter application functionality.

COMPLEX ENGINEERING PROBLEM - REPORT

6.3 Automated Validation and Merge

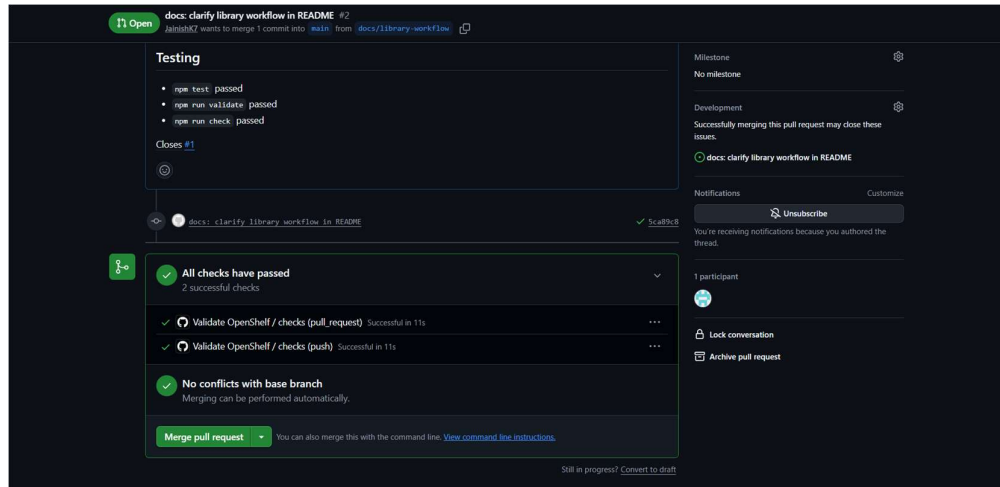


Figure 9 – GitHub Actions checks passed before merge

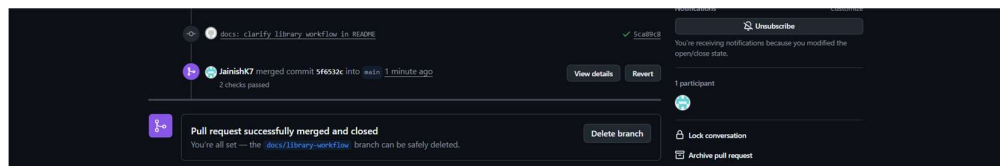


Figure 10 – Pull request successfully merged into main

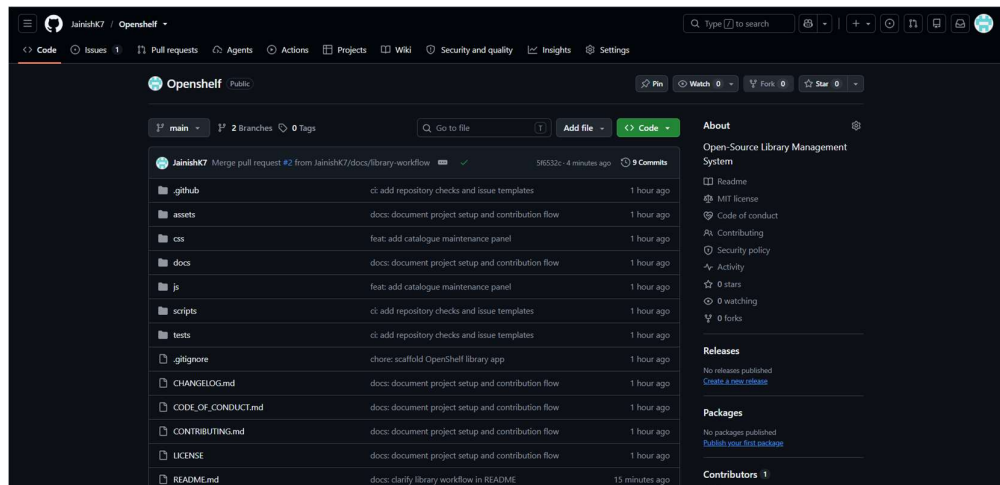


Figure 11 – Final main branch and completed repository structure

Two repository checks passed before merge, GitHub reported no conflicts, and the reviewed documentation change was merged into main. The final repository retains the application, documentation, testing, issue templates, and CI configuration.

COMPLEX ENGINEERING PROBLEM - REPORT

7. Testing and Results

Testing covered functional inspection, repository completeness, local validation, pull-request validation, and the final merged state. Repeatable Node.js checks were used in addition to manual inspection.

ID	Test / Parameter	Expected	Observed	Result
T01	Catalogue search/filter	Relevant records appear.	Search/filter controls are connected to catalogue rendering.	PASS
T02	Issue / return	Availability changes correctly.	Issue and return controls update catalogue activity.	PASS
T03	Persistence	State survives reload.	localStorage stores catalogue/activity state.	PASS
T04	Smoke test	Core hooks validate.	OpenShelf smoke test passed.	PASS
T05	Repository validation	Required files/checks present.	Repository validation passed.	PASS
T06	GitHub Actions	PR validation succeeds.	Two checks passed with no conflicts.	PASS
T07	Final merge	Reviewed change reaches main.	PR #2 merged and README retained.	PASS

7.1 Technical Analysis

Maintainability is supported by separation of application source, styles, tests, scripts, documentation, and contribution rules. Reliability is improved by repeatable validation and traceable Git history. The prototype avoids credentials and server-side secrets; a multi-user production system would require authenticated access, authorization, server-side validation, and protected data storage. Scalability is limited by browser-local persistence, so a production version should move shared records to a server-side API and database. The client-side scope keeps the current demonstration lightweight and easy to understand.

8. Challenges Faced and Solutions

Challenge	Solution
Keeping application and documentation consistent	Documented the project structure and workflow beside the actual source.
Avoiding direct changes on main	Used a dedicated documentation branch and pull request.
Making changes traceable	Linked Issue #1 to PR #2 and used a focused commit.
Checking repository completeness	Added repeatable smoke and repository-validation scripts.
Automating checks before merge	Configured GitHub Actions for push and pull-request validation.
Keeping the scope lightweight	Retained client-side scope and documented production limitations.

COMPLEX ENGINEERING PROBLEM - REPORT

9. Conclusion

The documentation and contribution workflow of OpenShelf was improved around a functioning library application. The repository now combines source code with structured documentation, contribution guidelines, issue templates, automated validation, and continuous integration. The README improvement was recorded through an issue, isolated on a branch, reviewed through a pull request, validated automatically, and merged into main. The final structure provides a clearer and more maintainable foundation for future contributors while preserving the project's lightweight scope.

10. SDG Mapping and Justification

SDG 4 – Quality Education: structured setup guidance, contribution rules, and practical software-development resources support hands-on learning.

SDG 9 – Industry, Innovation and Infrastructure: version control, modular organization, automated testing, continuous integration, and traceable change management are applied to the repository.

SDG 17 – Partnerships for the Goals: issues, branches, pull requests, contribution guidance, and automated checks establish a shared workflow for collaborative development.

COMPLEX ENGINEERING PROBLEM - REPORT

11. References (in IEEE Style)

- [1] Git, "Git Documentation," Software Freedom Conservancy, 2026. [Online]. Available: <https://git-scm.com/docs/git>
- [2] GitHub, "About issues," GitHub Docs, 2026. [Online]. Available: <https://docs.github.com/en/issues/tracking-your-work-with-issues/learning-about-issues/about-issues>
- [3] GitHub, "About pull requests," GitHub Docs, 2026. [Online]. Available: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
- [4] GitHub, "Understanding GitHub Actions," GitHub Docs, 2026. [Online]. Available: <https://docs.github.com/en/actions/get-started/understand-github-actions>
- [5] GitHub, "About the repository README file," GitHub Docs, 2026. [Online]. Available: <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>
- [6] Mozilla, "Window: localStorage property," MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- [7] Open Source Initiative, "The MIT License," 2026. [Online]. Available: <https://opensource.org/license/mit/>

COMPLEX ENGINEERING PROBLEM - REPORT

Appendix A – Final Repository Structure

Repository Area	Contents
.github/	ISSUE_TEMPLATE/ (bug_report.md, feature_request.md); PULL_REQUEST_TEMPLATE.md; workflows/validate.yml
assets/	architecture.svg
css/	styles.css
docs/	architecture.md; development.md; documentation-matrix.md; setup.md; workflow.md
js/	app.js
scripts/	validate-repo.mjs
tests/	smoke-test.mjs
Application root	index.html; package.json
Repository documentation	README.md; CONTRIBUTING.md; CODE_OF_CONDUCT.md; SECURITY.md; CHANGELOG.md; LICENSE; .gitignore

Appendix B – Final Commit History

5f6532c Merge pull request #2 from JainishK7/docs/library-workflow
 5ca89c8 docs: clarify library workflow in README
 813c318 ci: add repository checks and issue templates
 1498a9c docs: document project setup and contribution flow
 181b086 feat: add catalogue maintenance panel
 7c4dac0 feat: persist catalogue data locally
 b27187a feat: add issue and return tracking
 be6cb9e feat: add searchable library catalogue
 3b6c864 chore: scaffold OpenShelf library app

Appendix C – Photos

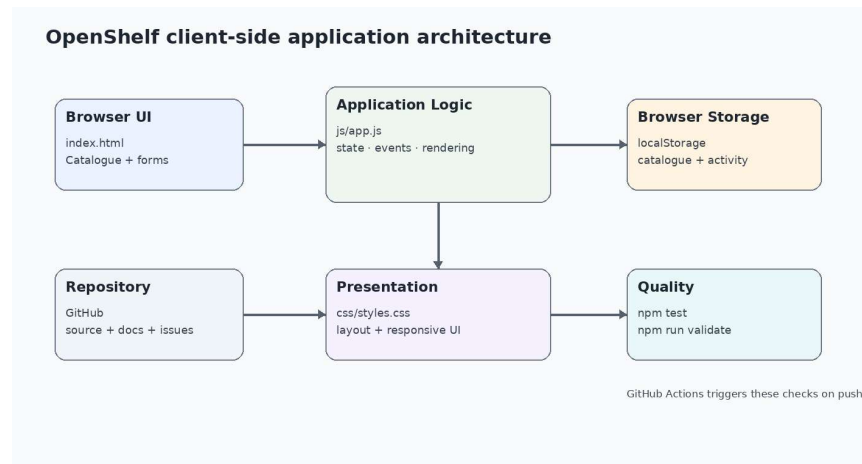


Figure 1 – OpenShelf client-side application architecture

COMPLEX ENGINEERING PROBLEM - REPORT



Figure 2 – Repository documentation improvement workflow

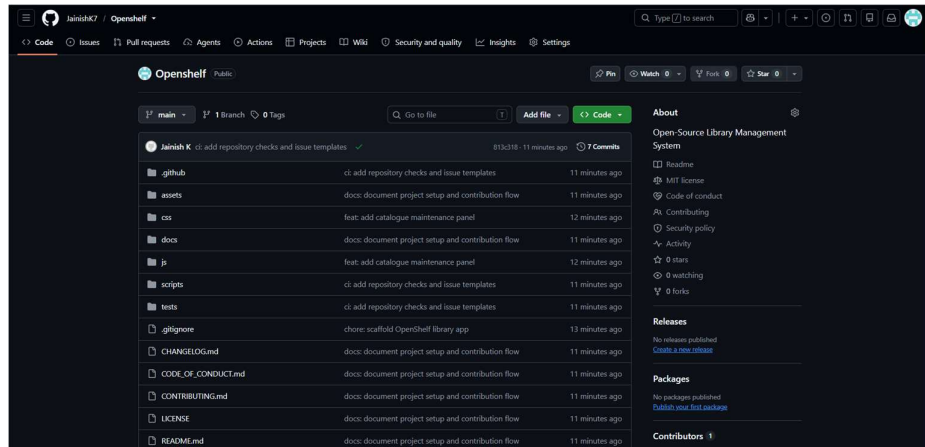


Figure 4 – OpenShelf repository baseline and project structure

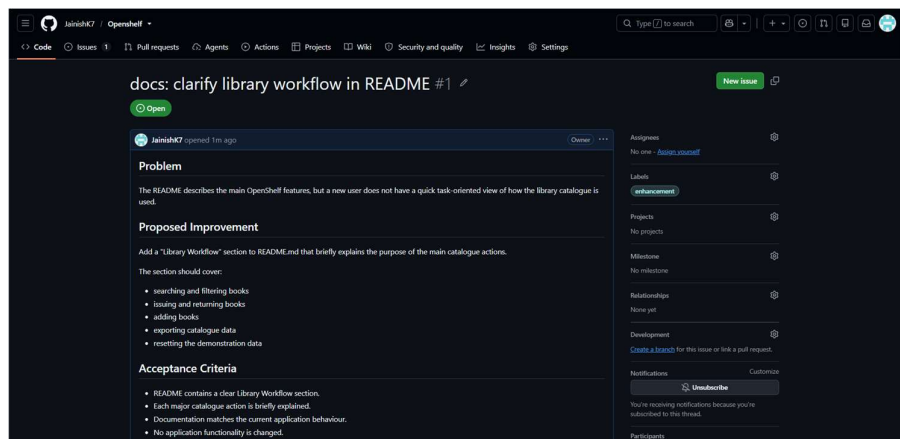


Figure 5 – GitHub Issue #1 documenting the README improvement

COMPLEX ENGINEERING PROBLEM - REPORT

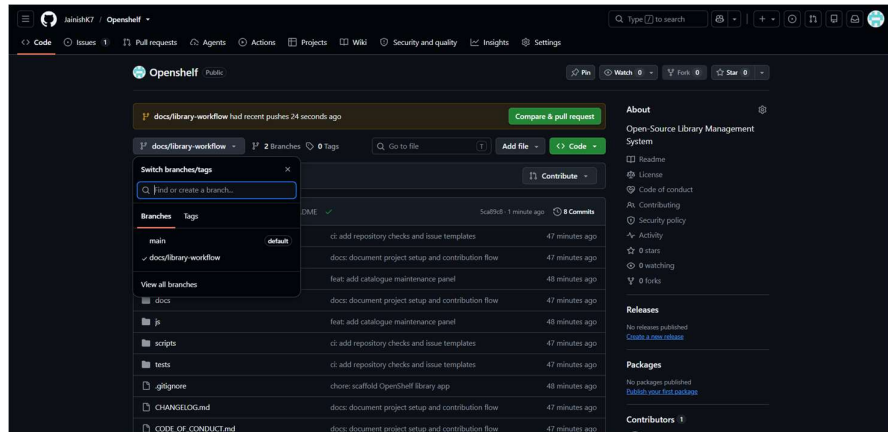


Figure 6 – docs/library-workflow branch

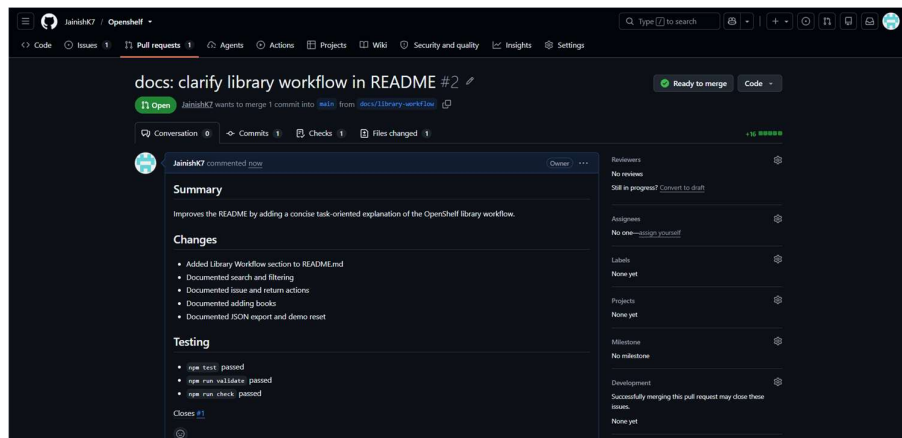


Figure 7 – Pull Request #2 for the README improvement

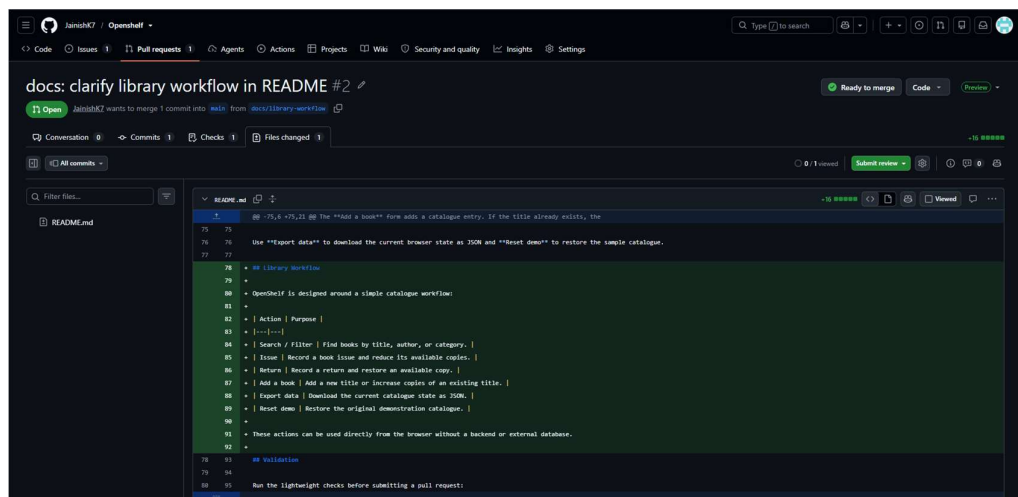


Figure 8 – README diff reviewed through Pull Request #2

COMPLEX ENGINEERING PROBLEM - REPORT

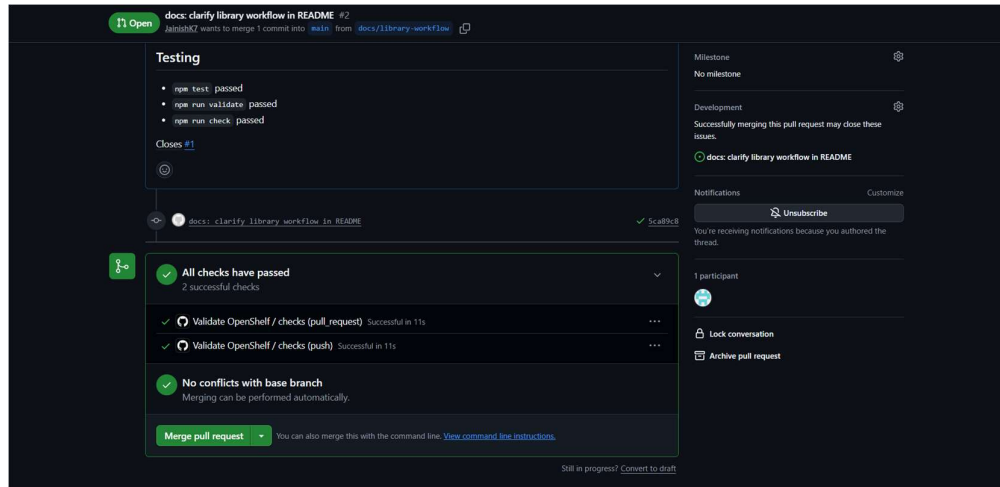


Figure 9 – GitHub Actions checks passed before merge

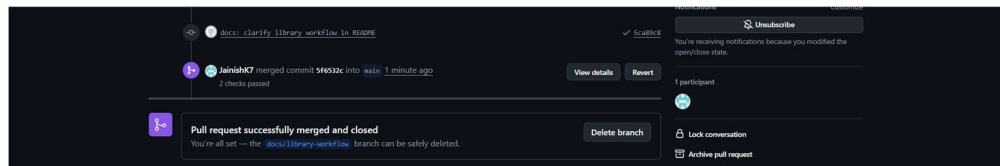


Figure 10 – Pull request successfully merged into main

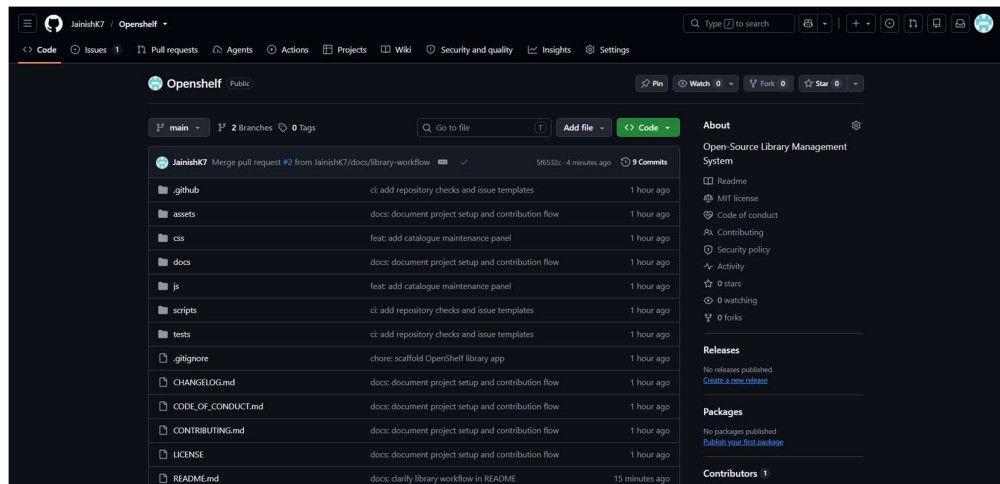


Figure 11 – Final main branch and completed repository structure